Experiment 'p_6_2_i_has_adjacent_duplicates_array' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_2_i_has_adjacent_duplicates_array
Natural language statement: Write a method to return true if the given array contains two adjacent duplicate elements.
Method signature: p_6_2_i_has_adjacent_duplicates_array(inputs: array<int>) returns (has_duplicates: bool)

Ensures

- `has_duplicates == (exists i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1])`

Requires

- `inputs.Length >= 2`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to return true if the given array contains two adjacent duplicate elements
    ↪ .

The signature should be:

p_6_2_i_has_adjacent_duplicates_array(inputs: array<int>) returns (has_duplicates: bool)

The method should respect the following contract:

requires inputs.Length >= 2, ensures has_duplicates == (exists i :: 0 <= i < inputs.
    ↪ Length - 1 && inputs[i] == inputs[i + 1])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn54dSgEQjjdZp7NKzxDF8XoqtLhx
Model: gpt-5.5-2026-04-23
Created at: 1780589947
Finish reason: stop
Usage: 326 tokens in, and 5420 tokens out

```
{
  var i := 0;
  has_duplicates := false;

  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length - 1
    invariant has_duplicates ==> (exists j :: 0 <= j < i && inputs[j] == inputs[j + 1])
    invariant forall j :: 0 <= j < i ==> inputs[j] != inputs[j + 1] || has_duplicates
    decreases inputs.Length - 1 - i
  {
    assert i + 1 < inputs.Length;

    if inputs[i] == inputs[i + 1] {
      has_duplicates := true;
      assert 0 <= i < i + 1 && inputs[i] == inputs[i + 1];
      assert exists j :: 0 <= j < i + 1 && inputs[j] == inputs[j + 1];
      assert forall j | 0 <= j < i + 1 ensures inputs[j] != inputs[j + 1] ||
          ↪ has_duplicates {
      }
```

```
    } else {
      if has_duplicates {
        var k :| 0 <= k < i && inputs[k] == inputs[k + 1];
        assert 0 <= k < i + 1 && inputs[k] == inputs[k + 1];
        assert exists j :: 0 <= j < i + 1 && inputs[j] == inputs[j + 1];
      }
      assert forall j | 0 <= j < i + 1 ensures inputs[j] != inputs[j + 1] ||
          ↪ has_duplicates {
        if j < i {
          assert inputs[j] != inputs[j + 1] || has_duplicates;
        } else {
          assert j == i;
          assert inputs[j] != inputs[j + 1];
        }
      }
    }

    i := i + 1;
  }

  assert i == inputs.Length - 1;

  if has_duplicates {
    var k :| 0 <= k < i && inputs[k] == inputs[k + 1];
    assert 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1];
    assert exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1];
    assert has_duplicates == (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] ==
        ↪ inputs[j + 1]);
  } else {
    assert forall j | 0 <= j < inputs.Length - 1 ensures inputs[j] != inputs[j + 1] {
      assert j < i;
      assert inputs[j] != inputs[j + 1] || has_duplicates;
    }
    if exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] {
      var k :| 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1];
      assert inputs[k] != inputs[k + 1];
      assert false;
    }
    assert !(exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]);
    assert has_duplicates == (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] ==
        ↪ inputs[j + 1]);
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(22,4): Error: invalid
    ↪ UnaryExpression
   |
```

```
22 | } else {
   | ^

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(21,6): Error: expected
    ↪ either 'by' or a semicolon following the assert expression
   |
21 | }
   | ^

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(36,4): Error: invalid
    ↪ UnaryExpression
   |
36 | }
   | ^

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(35,6): Error: expected
    ↪ either 'by' or a semicolon following the assert expression
   |
35 | }
   | ^

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(53,76): Error: then
    ↪ expected
   |
53 | if exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] {
   | ^

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(57,4): Error: invalid
    ↪ UnaryExpression
   |
57 | }
   | ^

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(60,2): Error: invalid
    ↪ UnaryExpression
   |
60 | }
   | ^

problem_data/p_6_2_i_has_adjacent_duplicates_array/program.dfy(59,99): Error: expected
    ↪ either 'by' or a semicolon following the assert expression
   |
59 | assert has_duplicates == (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] ==
    ↪ inputs[j + 1]);
   | ^

8 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn56U1Eq7MfSFJoUDSwp4x7QLdjvR
Model: gpt-5.5-2026-04-23
Created at: 1780590062
Finish reason: stop
Usage: 1880 tokens in, and 3087 tokens out

```
{
  var i := 0;

  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length - 1
    invariant forall j :: 0 <= j < i ==> inputs[j] != inputs[j + 1]
    decreases inputs.Length - 1 - i
  {
    if inputs[i] == inputs[i + 1] {
      has_duplicates := true;
      assert 0 <= i < inputs.Length - 1;
      assert inputs[i] == inputs[i + 1];
      assert exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1];
      return;
    } else {
      assert inputs[i] != inputs[i + 1];
      forall j | 0 <= j < i + 1
        ensures inputs[j] != inputs[j + 1]
      {
        if j < i {
          assert inputs[j] != inputs[j + 1];
        } else {
          assert j == i;
          assert inputs[j] != inputs[j + 1];
        }
      }
      i := i + 1;
    }
  }

  has_duplicates := false;
  assert i == inputs.Length - 1;

  forall j | 0 <= j < inputs.Length - 1
    ensures inputs[j] != inputs[j + 1]
  {
    assert j < i;
    assert inputs[j] != inputs[j + 1];
  }

  if (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]) {
    var k :| 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1];
    assert inputs[k] != inputs[k + 1];
```

```
    assert false;
  }

  assert !(exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]);
}
```

Final Program

```
method p_6_2_i_has_adjacent_duplicates_array(inputs: array<int>) returns (has_duplicates:
    ↪  bool)
      requires inputs.Length >= 2
      ensures has_duplicates == (exists i :: 0 <= i < inputs.Length - 1 && inputs[i] ==
          ↪ inputs[i + 1])
{
  var i := 0;

  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length - 1
    invariant forall j :: 0 <= j < i ==> inputs[j] != inputs[j + 1]
    decreases inputs.Length - 1 - i
  {
    if inputs[i] == inputs[i + 1] {
      has_duplicates := true;
      assert 0 <= i < inputs.Length - 1;
      assert inputs[i] == inputs[i + 1];
      assert exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1];
      return;
    } else {
      assert inputs[i] != inputs[i + 1];
      forall j | 0 <= j < i + 1
        ensures inputs[j] != inputs[j + 1]
      {
        if j < i {
          assert inputs[j] != inputs[j + 1];
        } else {
          assert j == i;
          assert inputs[j] != inputs[j + 1];
        }
      }
      i := i + 1;
    }
  }

  has_duplicates := false;
  assert i == inputs.Length - 1;

  forall j | 0 <= j < inputs.Length - 1
    ensures inputs[j] != inputs[j + 1]
  {
    assert j < i;
```

```
    assert inputs[j] != inputs[j + 1];
  }

  if (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]) {
    var k :| 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1];
    assert inputs[k] != inputs[k + 1];
    assert false;
  }

  assert !(exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]);
}
```

Total Token Usage

Input tokens: 2206
Output tokens: 8507
Reasoning tokens: 7196
Sum of 'total tokens': 10713

Experiment Timings

Overall Experiment started at 1780589943134, ended at 1780590169326, lasting 226192ms (226.19 seconds)
Iteration #1 started at 1780589943143, ended at 1780590061826, lasting 118683ms (118.68 seconds)
Iteration #2 started at 1780590061828, ended at 1780590169326, lasting 107498ms (107.50 seconds)